ARIEMEDI

精准导航　智引未来

瑞瞳 SDK 开发说明文档

Windows C#版

版本号 V4.8.0

型号规格 RUITONG SDK

艾瑞迈迪医疗科技（北京）有限公司

目录

1. 简介 5
1.1 产品概述 5
1.2 环境要求 5
1.2.1 系统要求 5
1.2.2 开发环境与编译语言 5
2. SDK 基础接口说明 5
2.1 设备连接接口 5
2.1.1 autoDeviceScan 5
2.1.2 getScannedDevice 5
2.1.3 connectDevice 6
2.1.4 disconnectDevice 6
2.1.5 getDeviceInfo 6
2.2 器械设置接口 7
2.2.1 loadPassiveToolAROM 7
2.2.2 getTrackToolsNum 7
2.2.3 getTrackToolsName 7
2.2.4 generateAROM 8
2.2.5 saveAROM 8
2.3 数据传输接口 9
2.3.1 setTrackingDataTransmissionType 9
2.3.2 startTracking 9
2.3.3 stopTracking 9
2.3.4 startImaging 10
2.3.5 stopImaging 10
2.3.6 startMonitoring 10
2.3.7 stopMonitoring 11
2.3.8 getTrackingToolCurrentNum 11
2.3.9 getSelectedTrackingData 11
2.3.10 getSelectedMarkerData 11
2.3.11 getAllMarkersNum 12
2.3.12 getPassiveMarkersNum 12
2.3.13 getActiveMarkersNum 12
2.3.14 getMarker 13
2.3.15 getUnMatchedMarkersNum 13
2.3.16 getUnMatchedMarker 13
2.3.17 trackingUpdate 14
2.3.18 toolUpdate 14
2.3.19 monitoringUpdate 14
2.3.20 getLeftImagingData 15
2.3.21 getRightImagingData 15
2.3.22 getIFImageSize 15
2.3.23 getMonitorImagingData 15

2.3.24 getMonitorImageSize 16
2.3.25 setMonitorImageResolution 16
2.4 状态监测接口 16
2.4.1 getConnectionStatus 16
2.4.2 getSystemAlert 17
2.4.3 getGravityVector 17
2.5 常用工具接口 17
2.5.1 pivotTipCalibration 18
2.5.2 fixedTipCalibration 18
2.5.3 fixedDirCalibration 19
2.5.4 setAreaDisplay 19
2.5.5 setAreaHidden 20
2.5.6 beep 20
2.5.7 laser 20
2.5.8 laserOn 21
2.5.9 getNetAdaptor 21
2.5.10 saveLog 21
3. SDK 重建接口说明 22
3.1 setMaterial 22
3.2 reconstruction 22
3.3 getPointCloud 22
3.4 getPointNum 23
3.5 getTextureImageUChar 23
3.6 getDepthImage 23
3.7 getDepthImageSize 24
3.8 project2DImagePosition 24
3.9 savePointCloud 24
4. SDK 重要数据结构 25
4.1 跟踪信息数据结构 TrackingData 25
4.2 标志物数据结构 MarkerPosition 25
4.3 方向向量数据结构 DirVector 25
4.4 器械标定数据结构 ToolInf 26
4.5 点（云）数据结构 PointInf 26
4.6 连接网卡信息数据结构 NetAdaptorInfo 26
4.7 连接设备信息数据结构 RTDeviceInfo 27
5. Demo 使用指南 27
5.1 连接准备 27
5.2 文件目录 27
5.3 工程配置 27
5.4 基础功能使用 27
5.5 高级功能使用 28
6. 常见问题 30

1. 简介

1.1 产品概述

适用于瑞瞳系列产品（瑞瞳 SE、瑞瞳 LITE、瑞瞳 MAX、瑞瞳 MAXV、瑞瞳 PRO）的 SDK 工具包，包含瑞瞳的连接与初始化、标志物定位、器械识别跟踪、表面点云重建等功能，用户可根据业务需求结合 SDK 灵活的进行应用层开发。

1.2 环境要求

1.2.1 系统要求

Windows 10/11 系统

1.2.2 开发环境与编译语言

Visual Studio 2019/2022, C#

2. SDK 基础接口说明

2.1 设备连接接口

功能包括：连接、断开、获取设备信息。

2.1.1 autoDeviceScan

原型

```
void autoDeviceScan(ref int num);
```

功能描述

自动扫描局部网内可连接的设备数量。

参数

ref int 类型：存放扫描到的局部网内可连接设备数量。

返回值

无。

2.1.2 getScannedDevice

原型

```
void getScannedDevice(int i, StringBuilder hostname, StringBuilder ip);
```

功能描述

获取自动扫描到的指定编号的局部网内可连接设备 hostname 和 IP 地址。

参数

① int 类型：指定设备编号，从 0 开始；

② StringBuilder 类型：用于存放指定设备的 hostname；

③ StringBuilder 类型：用于存放指定设备的 IP 地址。

返回值

无。

2.1.3 connectDevice

原型

```
int connectDevice(string hostname/IP);
```

功能描述

连接与初始化设备。

参数

string 类型：设备对应的 hostname 或 IP。

返回值

int 类型：连接设备成功或失败状态值，其中-1 对应“连接失败”，0 对应“连接成功”。

2.1.4 disconnectDevice

原型

```
void disconnectDevice();
```

功能描述

断开与设备的连接。

参数

无。

返回值

无。

2.1.5 getDeviceInfo

原型

```
RTDeviceInfo getDeviceInfo();
```

功能描述

获取已连接的设备信息。

参数

无。

返回值

RTDeviceInfo 类型：已连接的设备信息。

2.2 器械设置接口

功能包括：导入\生成\保存器械标定文件、获取器械数量。

2.2.1 loadPassiveToolAROM

原型

```
void loadPassiveToolAROM(string directory);
```

功能描述

从指定文件夹目录读取所有器械标定文件（*.arom）。

参数

string 类型：器械标定文件所在文件夹 directory。

返回值

无。

2.2.2 getTrackToolsNum

原型

```
int getTrackToolsNum();
```

功能描述

获取待跟踪所有器械数量。

参数

无。

返回值

int 类型：待跟踪器械数量，其与读入的器械标定文件（*.arom）数量一致。

2.2.3 getTrackToolsName

原型

```
void getToolName(int index, StringBuilder str, int strLen);
```

功能描述

获取指定器械名称。

参数

① int 类型：指定器械的编号，其范围为 0~(所有跟踪器械数量-1)；

② StringBuilder 类型：用于存放指定器械的名称；

③ int 类型：指定器械名称的长度。

返回值

无。

2.2.4 generateAROM

原型

ushort generateAROM(string directory, ToolInf tool, int[] markerPlaneNum, DirVector[] markerPlaneDir, MarkerPosition[] markers, MarkerPosition[] points, double radius = 5);

功能描述

根据用户输入的参数，生成器械标定文件（*.arom）。**注意**：该接口函数无法替代器械尖端标定过程，若要确定器械尖端坐标，请使用标定软件或调用 pivotTipCalibration 函数。

参数

① string 类型：生成路径；

② ToolInf 类型：器械信息结构体；

③ int[]类型：器械各平面内标志物数量；

④ DirVector[]类型：器械各平面法向量；

⑤ MarkerPosition[]类型：器械所有平面标志物信息，各平面内标志物依次排列；

⑥ MarkerPosition[]类型：器械所有虚拟点信息；

⑦ double 类型：标志物半径，单位：毫米。

返回值

ushort 类型：生成器械标定文件（*.arom）成功或失败信息，其中 0 为成功；1 为生成失败，最少匹配标志物数量过少；2 为生成失败，最少匹配标志物数量过多；3 为生成失败，平面数量过少；4 为生成失败，平面数量过多；5 为生成失败，平面数量不匹配；6 为生成失败，平面内标志物数量过少；7 为生成失败，平面内标志物数量过多；8 为生成失败，器械标志物数量过多；9 为生成失败，标志物间距过小；10 为生成失败，文件名包含中文与符号；11 为生成失败，文件存储路径不存在；12 为生成失败，文件类型错误。

2.2.5 saveAROM

原型

bool saveAROM(string directory, string toolName);

功能描述

将已导入的器械标定信息导出为标定文件（*.arom）。

参数

① string 类型：生成路径；

② string 类型：器械名称。

返回值

bool 类型：生成器械标定文件（*.arom）成功或失败。

2.3 数据传输接口

功能包括：刷新图像和标志物跟踪数据、获取图像与器械跟踪数据。

2.3.1 setTrackingDataTransmissionType

原型

```
void setTrackingDataTransmissionType(int type);
```

功能描述

设置传输跟踪数据的标志物类型。

参数

int 类型：标志物类型，其中 1 为只传输被动反光标志物跟踪数据，2 为只传输主动发光标志物跟踪数据，3 为混合不区分地发送被动反光与主动发光标志物跟踪数据，4 为间隔区分地发送被动反光与主动发光标志物跟踪数据。**注意**：若不设置，瑞瞳设备默认只传输被动反光标志物跟踪数据，且在混合传输模式下无法区分主动反光与被动发光标志物跟踪数据。

返回值

无。

2.3.2 startTracking

原型

```
void startTracking();
```

功能描述

开始跟踪器械。

参数

无。

返回值

无。

2.3.3 stopTracking

原型

```
void stopTracking();
```

功能描述

停止跟踪器械。

参数

无。

返回值

无。

2.3.4 startImaging

原型

```
void startImaging();
```

功能描述

开始传输双目红外图像。

参数

无。

返回值

无。

2.3.5 stopImaging

原型

```
void stopImaging();
```

功能描述

停止传输双目红外图像。

参数

无。

返回值

无。

2.3.6 startMonitoring

原型

```
void startMonitoring(bool r);
```

功能描述

开始传输可见光图像数据，图像默认为高分辨率尺寸 1920*1200。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

bool 类型：true 为高分辨率（1920*1200），false 为低分辨率（1280*720）。

返回值

无。

2.3.7 stopMonitoring

原型

void stopMonitoring();

功能描述

停止传输可见光图像数据。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

无。

2.3.8 getTrackingToolCurrentNum

原型

int getTrackingToolCurrentNum();

功能描述

获取当前跟踪的所有器械数量。

参数

无。

返回值

int 类型：当前跟踪的所有器械数量。

2.3.9 getSelectedTrackingData

原型

TrackingData getSelectedTrackingData(int index);

功能描述

获取指定器械的跟踪数据。

参数

int 类型：指定器械的编号，其范围为 0~(所有跟踪器械数量-1)。

返回值

TrackingData 类型：指定器械的跟踪数据，包括跟踪状态、器械位姿、跟踪误差等。该类型信息详见 4.1 节。

2.3.10 getSelectedMarkerData

原型

MarkerPosition getSelectedMarkerData (int indexT, int indexM);

功能描述

获取指定器械的指定标志物位置信息。

参数

① int 类型：指定器械的编号，其范围为 0~(所有跟踪器械数量-1)；

② int 类型：指定标志物编号，其范围为 0~(该器械上所有标志物数量-1)。

返回值

TrackingData 类型：指定器械的指定标志物位置信息，包括 x，y，z 坐标。该类型信息详见 4.2 节。

2.3.11 getAllMarkersNum

原型

int getAllMarkersNum();

功能描述

获取当前视场中所有标志物的数量。

参数

无。

返回值

int 类型：当前视场中所有标志物的数量。

2.3.12 getPassiveMarkersNum

原型

int getPassiveMarkersNum();

功能描述

获取当前视场中所有被动反光标志物的数量。

参数

无。

返回值

int 类型：当前视场中所有被动反光标志物的数量。

2.3.13 getActiveMarkersNum

原型

int getActiveMarkersNum();

功能描述

获取当前视场中所有主动发光标志物的数量。

参数

无。

返回值

int 类型：当前视场中所有主动发光标志物的数量。

2.3.14 getMarker

原型

MarkerPosition getMarker(int index);

功能描述

获取当前视场中指定标志物坐标。

参数

int 类型：指定标志物的编号，其范围为 0~(所有标志物数量-1)。

返回值

MarkerPosition 类型：指定标志物的坐标。该类型信息详见 4.2 节。

2.3.15 getUnMatchedMarkersNum

原型

int getUnMatchedMarkersNum();

功能描述

获取当前所有游离的未匹配标志物数量。该函数需在 getTrackingData 后运行，才可以获取正确数量。

参数

无。

返回值

int 类型：当前视场中所有游离的未匹配标志物数量。

2.3.16 getUnMatchedMarker

原型

MarkerPosition getUnMatchedMarker(int index);

功能描述

获取当前指定游离的未匹配标志物坐标。该函数需在 getTrackingData 后运行，才可以获取正确数据。

参数

int 类型：指定标志物的编号，其范围为 0~(所有标志物数量-1)。

返回值

MarkerPosition 类型：指定游离的未匹配标志物坐标。该类型信息详见 4.2 节。

2.3.17 trackingUpdate

原型

```
void trackingUpdate();
```

功能描述

刷新所有标志物的跟踪数据。

参数

无。

返回值

无。

2.3.18 toolUpdate

原型

```
void toolUpdate();
```

功能描述

刷新所有器械的跟踪数据。

参数

无。

返回值

无。

2.3.19 monitoringUpdate

原型

```
void monitoringUpdate();
```

功能描述

刷新可见光图像数据。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

无。

2.3.20 getLeftImagingData

原型

```
IntPtr getLeftImagingData();
```

功能描述

获取当前左目相机图像，图像尺寸为 1280*800 像素（SE、LITE、MAX 与 MAXV 型号）或 1920*1200 像素（PRO 型号）。

参数

无。

返回值

IntPtr 类型：当前左目相机图像数据。

2.3.21 getRightImagingData

原型

```
IntPtr getRightImagingData();
```

功能描述

获取当前右目相机图像，图像尺寸为 1280*800 像素（SE、LITE、MAX 与 MAXV 型号）或 1920*1200 像素（PRO 型号）。

参数

无。

返回值

IntPtr 类型：当前右目相机图像数据。

2.3.22 getIFImageSize

原型

```
void getIFImageSize(ref int size);
```

功能描述

获取双目相机图像尺寸。

参数

ref int 类型：返回双目相机图像长宽。

返回值

无。

2.3.23 getMonitorImagingData

原型

IntPtr getMonitorImagingData();

功能描述

获取当前可见光相机图像，高分辨率图像尺寸为 1920*1200 像素，低分辨率图像尺寸为 1280*720 像素。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

IntPtr 类型：当前可见光相机图像数据。

2.3.24 getMonitorImageSize

原型

void getMonitorImageSize(ref int size);

功能描述

获取可见光相机图像尺寸。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

ref int 类型：返回可见光相机图像长宽。

返回值

无。

2.3.25 setMonitorImageResolution

原型

void setMonitorImageResolution(bool r);

功能描述

设置可见光相机图像分辨率。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号，若不设置默认为高分辨率。

参数

bool 类型：false 为低分辨率（1280*720），true 为高分辨率（1920*1200）。

返回值

无。

2.4 状态监测接口

功能包括：获取设备连接状态、系统警告、重力向量。

2.4.1 getConnectionStatus

原型

ushort getConnectionStatus();

功能描述

获取当前设备连接状态。

参数

无。

返回值

ushort 类型：当前设备连接状态，其中 0 为连接成功，1 为断开成功；2 为断开失败；3 为连接失败，连接端口冲突；4 为连接失败，IP 解析错误；5 为连接失败，初始化错误；6 为连接中断；7 为连接失败，IP 地址无效；8 为连接失败，设备不存在；9 为连接失败，设备连接超时；10 为连接失败，未知错误。

2.4.2 getSystemAlert

原型

ushort getSystemAlert();

功能描述

获取当前设备警告信息。

参数

无。

返回值

ushort 类型：当前设备温度、震动警告信息，其中 0 为无警告；1 为震动警告，剧烈震动已影响设备精度，建议返厂；2 为高温警告，高温已影响设备精度，请为设备降温；3 为低温警告，低温已影响设备精度，请提升环境温度。

2.4.3 getGravityVector

原型

DirVector getGravityVector();

功能描述

获取设备重力向量（单位向量）。

参数

无。

返回值

DirVector 类型：设备重力向量。该类型信息详见 4.3 节。

2.5 常用工具接口

功能包括：标定器械尖端坐标、在双目红外图像上绘制重建的矩形区域、设备蜂鸣、开启定位激光。

2.5.1 pivotTipCalibration

原型

ushort pivotTipCalibration(string toolname, TrackingData[] trackingdataVect, int dataNum, ref double tip, ref double offset, ref double error, bool update = false);

功能描述

根据器械绕尖端点转动的跟踪数据标定其在自身坐标系下的尖端坐标。

参数

① string 类型：待标定器械名称；

② TrackingData[]类型：跟踪数据；

③ int 类型：跟踪数据数量；

④ ref double 类型：尖端坐标标定结果；

⑤ ref double 类型：当前与上次标定结果在器械坐标系下 x，y，z 方向上的偏差；

⑥ ref double 类型：用于存储标定误差，若标定失败，返回-1；

⑦ bool 类型：是否将跟踪器械的尖端坐标更新

返回值

ushort 类型：标定器械尖端坐标成功或失败信息，其中 0 为标定成功；1 为标定失败，输入的器械跟踪数据异常；2 为标定失败，输入的器械跟踪数据不足；3 为标定失败，标定过程计算异常。

2.5.2 fixedTipCalibration

原型

ushort fixedTipCalibration(string toolname, TrackingData trackingdata, MarkerPosition point, ref double tip, ref double offset, bool update = false);

功能描述

根据器械的当前跟踪数据与瑞瞳坐标系下尖端点坐标标定其在自身坐标系下的尖端坐标。

参数

① string 类型：待标定器械名称；

② TrackingData 类型：跟踪数据；

③ MarkerPosition 类型：瑞瞳坐标系下尖端点坐标；

④ ref double 类型：尖端坐标标定结果；

⑤ ref double 类型：当前与上次标定结果在器械坐标系下 x，y，z 方向上的偏差；

⑥ bool 类型：是否将跟踪器械的尖端坐标更新

返回值

ushort 类型：标定器械尖端坐标成功或失败信息，其中 0 为标定成功；1 为标定失败，输入的器械跟踪数据异常。

2.5.3 fixedDirCalibration

原型

ushort fixedDirCalibration(string toolname, TrackingData trackingdata, ref double vec, ref double dir, bool update = false);

功能描述

根据器械的当前跟踪数据与瑞瞳坐标系下方向向量标定其在自身坐标系下的方向向量。

参数

① string 类型：待标定器械名称；

② TrackingData 类型：跟踪数据；

③ ref double 类型：瑞瞳坐标系下方向向量；

④ ref double 类型：方向标定结果；

⑤ bool 类型：是否将跟踪器械的方向更新

返回值

ushort 类型：标定器械尖端坐标成功或失败信息，其中 0 为标定成功；1 为标定失败，输入的器械跟踪数据异常。

2.5.4 setAreaDisplay

原型

void setAreaDisplay(short width, short height, short leftX, short leftY, short rightX, short rightY);

功能描述

在双目图像上绘制矩形框。

参数

① short 类型：绘制矩形区域长度 width（单位：像素）；

② short 类型：绘制矩形区域宽度 height（单位：像素）；

③ short 类型：绘制矩形区域在左目图像 x 方向上的起始坐标 leftX；

④ short 类型：绘制矩形区域在左目图像 y 方向上的起始坐标 leftY；

⑤ short 类型：绘制矩形区域在右目图像 x 方向上的起始坐标 rightX；

⑥ short 类型：绘制矩形区域在右目图像 y 方向上的起始坐标 rightY。

返回值

无。

2.5.5 setAreaHidden

原型

```
void setAreaHidden();
```

功能描述

隐藏在双目图像上绘制的矩形框。

参数

无。

返回值

无。

2.5.6 beep

原型

```
void beep(int delay);
```

功能描述

开始蜂鸣，且将在设定延迟时间后关闭。

参数

int 类型：延迟时间，单位：毫秒。

返回值

无。

2.5.7 laser

原型

```
void laser(bool s);
```

功能描述

开启/关闭定位激光。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

bool 类型：开启（true），关闭（false）。

返回值

无。

2.5.8 laserOn

原型

void laserOn(int delay);

功能描述

打开定位激光，且激光将在设定延迟时间后关闭。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

int 类型：延迟时间，单位：毫秒。

返回值

无。

2.5.9 getNetAdaptor

原型

NetAdaptorInfo getNetAdaptor();

功能描述

获取与设备连接的网卡信息。

参数

无。

返回值

NetAdaptorInfo 类型：包括网卡名称、连接方式、网速（单位：Mb/s）、连接状态。

2.5.10 saveLog

原型

bool saveLog(string path);

功能描述

保存日志信息。

参数

string 类型：保存路径。

返回值

bool 类型：保存成功与否。

3. SDK 重建接口说明

功能包括：在重建、保存点云。**注意**：该接口函数仅适用于 PRO 型号。

3.1 setMaterial

原型

```
void setMaterial(int material);
```

功能描述

设置重建物体的材质。注意：当前仅支持硅胶（0）与石膏材质（1），默认为硅胶材质。

参数

int 类型：材质编号。

返回值

无。

3.2 reconstruction

原型

```
void reconstruction(int width, int height, int leftX, int leftY, int rightX, int rightY);
```

功能描述

重建指定范围内的点云。

参数

① int 类型：重建区域长度 width（单位：像素）；

② int 类型：重建区域宽度 height（单位：像素）；

③ int 类型：重建区域在左目图像 x 方向上的起始坐标 leftX；

④ int 类型：重建区域在左目图像 y 方向上的起始坐标 leftY；

⑤ int 类型：重建区域在右目图像 x 方向上的起始坐标 rightX；

⑥ int 类型：重建区域在右目图像 y 方向上的起始坐标 rightY。

返回值

无。

3.3 getPointCloud

原型

```
PointInf getPointCloud(int index);
```

功能描述

获取点云中指定编号点的信息。

参数

int 类型：点编号。

返回值

PointInf 类型：点云中指定编号点的信息。该类型信息详见 4.4 节。

3.4 getPointNum

原型

int getPointNum();

功能描述

获取点云所有点数量。

参数

无。

返回值

int 类型：所有点数量。

3.5 getTextureImageUChar

原型

IntPtr getTextureImageUChar();

功能描述

获取点云纹理的可见光图像，其尺寸为 1920*1200。

参数

无。

返回值

IntPtr 类型：点云纹理的可见光图像。

3.6 getDepthImage

原型

void getDepthImage(float[] depthImage);

功能描述

获取点云重建的深度图。

参数

float[]类型：存放点云重建的深度图。

返回值

无。

3.7 getDepthImageSize

原型

void getDepthImageSize(ref int size);

功能描述

获取点云重建的深度图尺寸。

参数

ref int 类型：返回深度图长宽。

返回值

无。

3.8 project2DImagePosition

原型

int project2DImagePosition(int x, int y);

功能描述

获取可见光图像坐标对应的点（云）编号，其中图像坐标原点为左上角。

参数

① int 类型：可见光图像坐标 x（单位：像素）；

② int 类型：可见光图像坐标 y（单位：像素）。

返回值

int 类型：点（云）编号，可通过 getPointCloud()获取该编号点的坐标，若输入的可见光图像坐标无对应的点（云），则返回-1。

3.9 savePointCloud

原型

bool savePointCloud(string path, bool texture);

功能描述

将点云保存至指定路径。

参数

① string 类型：保存路径；

② bool 类型：是否写入纹理。

返回值

bool 类型：true 保存成功，false 保存失败。

4. SDK 重要数据结构

4.1 跟踪信息数据结构 TrackingData

```
struct TrackingData
{
    int matchStatus;        //匹配状态：1（跟踪），0（丢失）
    int matchedPlane;       //匹配平面编号
    int markerNum;          //器械上标志物数量
    int pointNum;           //虚拟点数量
    double tx;              //尖端坐标 x，单位：毫米
    double ty;              //尖端坐标 y，单位：毫米
    double tz;              //尖端坐标 z，单位：毫米
    double ox;              //姿态平移 x，单位：毫米
    double oy;              //姿态平移 y，单位：毫米
    double oz;              //姿态平移 z，单位：毫米
    double qw;              //姿态四元数 w
    double qx;              //姿态四元数 x
    double qy;              //姿态四元数 y
    double qz;              //姿态四元数 z
    double rxx;             //旋转矩阵(1,1)
    double ryx;             //旋转矩阵(1,2)
    double rzx;             //旋转矩阵(1,3)
    double rxy;             //旋转矩阵(2,1)
    double ryy;             //旋转矩阵(2,2)
    double rzy;             //旋转矩阵(2,3)
    double rxz;             //旋转矩阵(3,1)
    double ryz;             //旋转矩阵(3,2)
    double rzz;             //旋转矩阵(3,3)
    double dx;              //虚拟方向 x
    double dy;              //虚拟方向 y
    double dz;              //虚拟方向 z
    double error;           //匹配误差，单位：毫米
    double index;           //器械编号
}
```

4.2 标志物数据结构 MarkerPosition

```
struct MarkerPosition
{
    double x;               //标志物坐标 x，单位：毫米
    double y;               //标志物坐标 y，单位：毫米
    double z;               //标志物坐标 z，单位：毫米
    int type;               //标志物类型，0-未知，1-被动，2-主动
    bool RF;                //标志物识别结果，false-疑似，true-真实
}
```

4.3 方向向量数据结构 DirVector

```
struct DirVector
{
    double vx;                          //方向向量 x 方向分量
    double vy;                          //方向向量 y 方向分量
    double vz;                          //方向向量 z 方向分量
}
```

4.4 器械标定数据结构 ToolInf

```
struct ToolInf
{
    string name;                        //器械名称
    bool markerType;                    //标志物类型，false（被动），true（主动）
    int minNumMarker;                   //最少标志物匹配数量
    int planeNum;                       //平面数量
    int pointNum;                       //虚拟点数量
    double dirx;                        //虚拟方向向量 x 分量
    double diry;                        //虚拟方向向量 y 分量
    double dirz;                        //虚拟方向向量 z 分量
    double pinx;                        //尖端坐标 x 分量
    double piny;                        //尖端坐标 y 分量
    double pinz;                        //尖端坐标 z 分量
    double calbError;                   //标定误差
    double maxFRE;                      //最大匹配误差
    double maxAngle;                    //最大匹配角度
    int structureType;                  //器械结构：0 单平面，1 多平面，2 非平面
    int algorithmType;                  //匹配算法：0 平衡，1 精度优先，2 速度优先
}
```

4.5 点（云）数据结构 PointInf

```
struct PointInf
{
    float x;                            //点坐标 x，单位：毫米
    float y;                            //点坐标 y，单位：毫米
    float z;                            //点坐标 z，单位：毫米
    float r;                            //点纹理颜色 r 值，范围为 0-1
    float g;                            //点纹理颜色 g 值，范围为 0-1
    float b;                            //点纹理颜色 b 值，范围为 0-1
}
```

4.6 连接网卡信息数据结构 NetAdaptorInfo

```
struct NetAdaptorInfo
{
    string name;                        //网卡名称
    string connectionType;              //连接方式
    int speed;                          //传输速度，单位：Mb/s
    bool linked;                        //连接状态
```

```
}
```

4.7 连接设备信息数据结构 RTDeviceInfo

```
struct RTDeviceInfo
{
    string hostname;            //设备 hostname
    string IP;                  //设备 IP 地址
    string MAC;                 //设备 MAC 地址
    int deviceType;         //设备型号，0-未知，1-SE，2-LITE，3-MAX，4-MAXV，5-PRO
    int firmwareVersionL;       //设备固件版本号（第一位）
    int firmwareVersionM;       //设备固件版本号（第二位）
    int firmwareVersionH;       //设备固件版本号（第三位）
}
```

5. Demo 使用指南

SDK 的 Windows C#版本 Demo 使用指南以 Visual Studio 2019 与 Unity 3D 平台为例，其中测试该 Demo 的 Unity 3D 版本为 2019.4.4。

5.1 连接准备

查看接入瑞瞳设备背面标记的 hostname，例如 RT-SE000001.local。

5.2 文件目录

①Unity 3D 平台下，Plugins 文件夹包含 SDK 库文件（*.dll），Scripts 文件夹包含调用 SDK 的示例代码 DemoTest.cs，Tool 文件夹包含器械模型（*.obj）与对应的标定文件（*.arom）。

②Visual Studio 2019 平台下，除了库文件与标定文件，Program.cs 为调用 SDK 的示例代码。

5.3 工程配置

①Unity 3D 平台下，将 Plugins 与 Scripts 文件夹内容拷贝到 Unity 3D 项目里对应文件夹下，将 Tool 文件夹放置到 Plugins 与 Scripts 同级目录下。

②Visual Studio 2019 平台下，将库文件与 Tool 文件夹放置到 C#工程生成的 exe 同级目录下。

5.4 基础功能使用

①Unity 3D 平台下，基础功能包括连接、获取设备状态信息、获取标志物坐标、跟踪器械以及显示双目红外图像等，使用的 SDK 库文件包括 ARMDstereotracking.dll 与 ARMDCSDLLWarpper.dll，适用于瑞瞳所有型号。

主要函数功能介绍如下：

Start()：连接设备并初始化，读入器械标定文件，创建跟踪数据刷新线程

设置设备的 hostname，并连接，若连接成功，则创建跟踪数据刷新线程。读入 tool 路径下所有器械标定文件（*.arom）。

RTUpdate()：跟踪数据与双目红外图像数据刷新

循环更新当前双目红外图像以及所有标志物跟踪数据。

Update()：更新场景中器械状态与图像显示内容

根据跟踪数据，更新场景中器械位置、姿态与跟踪状态。根据双目红外图像数据，更新场景中图像区域显示内容。

OnDestory()：关闭系统，断开连接

停止跟踪，断开连接。

②Visual Studio 2019 平台下，基础功能使用与 Unity 3D 平台一致。

5.5 高级功能使用

①Unity 3D 平台，在基础功能上，高级功能包括点云重建、显示与保存点云以及显示可见光图像等，使用的 SDK 库文件包括 ARMDstereotracking.dll、ARMDstereoreconstruction.dll 与 ARMDCSDLLWarpperPro.dll 以及相应的 OpenCV 动态库文件，其中 Demo 使用的 OpenCV 版本为 4.5.2。除此之外，*.shader 文件用于显示点云纹理。高级功能仅适用于瑞瞳 PRO（点云+可见光）与瑞瞳 MAXV（可见光）

如图 1 所示，场景中用于显示交互的组件包括重建按钮 Button，双目红外图像显示窗口 LeftImage/Canvas/RawImage（图 1 左侧）与 RightImage/Canvas/RawImage（图 1 右侧），以及可见光图像显示窗口 CenterImage/Canvas/RawImage（图 1 中间）。

主要函数功能介绍如下：

Start()：连接设备并初始化，读入器械标定文件，创建跟踪数据刷新线程

设置设备的 hostname，并连接，若连接成功，则创建跟踪数据刷新线程。读入 Tool 路径下所有器械标定文件（*.arom）。

RTUpdateTracking()：跟踪数据与双目红外图像数据刷新

循环更新当前双目红外图像以及所有标志物跟踪数据。

RTUpdateMonitoring()：可见光图像数据刷新

循环更新当前可见光图像数据。

Update()：更新场景中器械状态与图像显示内容

根据跟踪数据，更新场景中器械位置、姿态与跟踪状态。根据双目红外图像与可见光图像数据，更新场景中图像区域（LeftImage/Canvas/RawImage、RightImage/Canvas/RawImage、CenterImage/Canvas/RawImage）显示内容。

Click()：点击重建按钮，执行点云重建

该函数需关联场景中的重建按钮，点击后点云会重建并显示在场景中，且保存到指定路径。

CreateMesh()：生成点云的 GameObject 组件

根据重建点云结果，生成包含点坐标与纹理信息的 GameObject 组件用于场景显示。

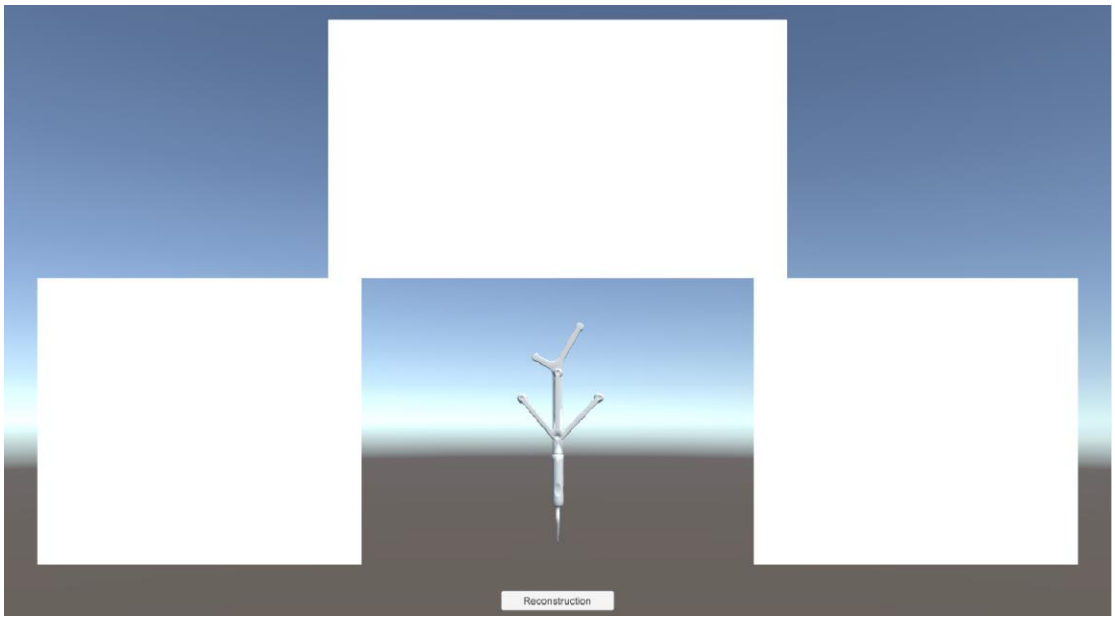

图 1 高级功能使用场景

②Visual Studio 2019 平台，在基础功能上，高级功能包括点云重建、显示与保存点云以及显示可见光图像等，使用的 SDK 库文件包括 ARMDstereotracking.dll、ARMDstereoreconstruction.dll 与 ARMDCSDLLWarpperPro.dll 以及相应的 OpenCV 动态库文件，其中 Demo 使用的 OpenCV 版本为 4.5.2。

主要函数功能介绍如下：

Start()：连接设备并初始化，读入器械标定文件，创建跟踪数据刷新线程

设置设备的 hostname，并连接，若连接成功，则创建跟踪数据刷新线程。读入 Tool 路径下所有器械标定文件（*.arom）。

RTUpdateTracking()：跟踪数据与双目红外图像数据刷新

循环更新当前双目红外图像以及所有标志物跟踪数据。

RTUpdateMonitoring()：可见光图像数据刷新

循环更新当前可见光图像数据。

ToolUpdate()：更新器械状态信息

更新器械跟踪数据与双目红外图像、可见光图像数据。

Reconstruction()：点云重建

重建指定范围内的表面点云，且保存到指定路径。

6. 常见问题

Q：启动系统时显示连接失败。

A：检查连接时输入的 hostname 是否与设备背面标记的一致。

Q：手动连接设备成功，但自动连接设备失败。

A：请关闭**所有**防火墙后，重新自动连接。